```
'use strict';
/**
 * Runner : lit les bougies de ton compte MetaApi, fait tourner le moteur MDT à c
 * clôture de bougie, et envoie chaque signal au webhook EXISTANT de ton site
 * (même JSON que TradingView, donc rien à changer côté exécution d'ordres).
 *
 * npm i metaapi.cloud-sdk
 * Variables d'environnement :
 *   METAAPI_TOKEN, METAAPI_ACCOUNT_ID   (ceux déjà utilisés par ton site)
 *   WEBHOOK_URL      URL affichée dans l'onglet "Signaux Webhook"
 *   WEBHOOK_SECRET   la clé secrète déjà attendue par ton serveur
 *   SYMBOL=GOLD      symbole exact chez ton courtier
 *   TIMEFRAME=15m    la même unité de temps que sur ton graphique TradingView
 */
const MetaApi = require('metaapi.cloud-sdk').default;
const { MdtEngine } = require('./mdtEngine');

const {
  METAAPI_TOKEN, METAAPI_ACCOUNT_ID, WEBHOOK_URL, WEBHOOK_SECRET,
  SYMBOL = 'GOLD', TIMEFRAME = '15m',
} = process.env;

const TF_MS = { '1m': 60e3, '5m': 300e3, '15m': 900e3, '30m': 1800e3, '1h': 3600e

const ema = (values, len) => {
  const k = 2 / (len + 1);
  let e = values[0];
  for (let i = 1; i < values.length; i++) e = values[i] * k + e * (1 - k);
  return e;
};

async function getCandles(account, tf, limit) {
  const r = await account.getHistoricalCandles(SYMBOL, tf, undefined, limit);
  return r
    .map((x) => ({
      time: new Date(x.time), open: x.open, high: x.high, low: x.low, close: x.cl
      volume: x.tickVolume ?? x.volume ?? 0,
    }))
    .sort((a, b) => a.time - b.time);
}

const closed = (tf) => (k) => k.time.getTime() + TF_MS[tf] <= Date.now();
```

```
async function getHtf(account) {
  const [h1, h4] = await Promise.all([getCandles(account, '1h', 500), getCandles(
  const c1 = h1.filter(closed('1h')).map((k) => k.close);
  const c4 = h4.filter(closed('4h')).map((k) => k.close);
  return { ema1: ema(c1, 200), ema2: ema(c4, 200) };
}

async function send(ev) {
  console.log('[MDT] signal', ev.action, JSON.stringify({ ...ev, secret: '***' })
  const res = await fetch(WEBHOOK_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(ev),
  });
  if (!res.ok) console.error('[MDT] webhook HTTP', res.status);
}

async function main() {
  if (!TF_MS[TIMEFRAME]) throw new Error('TIMEFRAME invalide : ' + TIMEFRAME);
  const api = new MetaApi(METAAPI_TOKEN);
  const account = await api.metatraderAccountApi.getAccount(METAAPI_ACCOUNT_ID);
  if (account.state !== 'DEPLOYED') await account.deploy();
  await account.waitConnected();

  const engine = new MdtEngine({ secret: WEBHOOK_SECRET });

  // Rejeu silencieux de l'historique (aucun ordre envoyé) pour chauffer les indi
  let htf = await getHtf(account);
  const hist = (await getCandles(account, TIMEFRAME, 1000)).filter(closed(TIMEFRA
  for (const k of hist) engine.onCandle(k, htf);
  engine.finishWarmup();
  let last = hist[hist.length - 1].time.getTime();
  console.log(`[MDT] prêt : ${hist.length} bougies ${TIMEFRAME} chargées, surveil

  let busy = false;
  setInterval(async () => {
    if (busy) return;
    busy = true;
    try {
      const fresh = (await getCandles(account, TIMEFRAME, 5))
        .filter(closed(TIMEFRAME))
        .filter((k) => k.time.getTime() > last);
      if (!fresh.length) return;
      htf = await getHtf(account);
      for (const k of fresh) {
        last = k.time.getTime();
```

```
        for (const ev of engine.onCandle(k, htf)) await send(ev);
      }
    } catch (e) {
      console.error('[MDT] erreur', e.message);
    } finally {
      busy = false;
    }
  }, 20000);
}

main().catch((e) => { console.error('[MDT] arrêt', e); process.exit(1); });
```